

Distributed Databases: Joins using MPI

Analysis of Barthels et al., *Distributed Join Algorithms on
Thousands of Cores*

Outline

- Introduction
- Distributed Sort-Merge Algorithm
- Sort-Merge using MPI
- Analysis

Outline

- **Introduction**
- Distributed Sort-Merge Algorithm
- Sort-Merge using MPI
- Analysis

Database Joins

Joins combine rows from two or more tables based on a join key.

```
SELECT *  
FROM Orders  
INNER JOIN Customers ON Orders.CustomerID=Customers.CustomerID;
```

Require join algorithms such as

- Nested Loop Join
- Sort-Merge Join
- Hash Join

Distributed Databases

Why we use distributed databases:

- To scale out and handle large volumes of data.
- To improve fault tolerance and availability through replication.
- To leverage physical data locality for better performance.
- To parallelize computation across multiple nodes for faster query processing.

The challenges:

- It is difficult to decide how to split data across nodes.
- The process must consider network communication overhead of data transfer.

Distributed Joins

Why are distributed joins challenging?

- Often require data shuffling between nodes, which is costly.
- Need to ensure that matching keys appear on the same node to get correct results.
- Require coordinating data movement, memory access, and merging between nodes / processes.

Barthels et al., *Distributed Join Algorithms on Thousands of Cores*

- How join algorithms behave at scales normally used for massive parallel computation.
 - Radix Hash Join
 - Sort-Merge Join
- Implementation of above algorithms with MPI.
- Performance evaluation on the range of 128 cores to 4096 cores.

Outline

- Introduction
- **Distributed Sort-Merge Algorithm**
- Sort-Merge using MPI
- Analysis

Distributed Sort-Merge Algorithm Steps

1. Partition within processes

2. Sort locally

3. Exchange across nodes

4. Merge sorted runs

5. Perform local joins

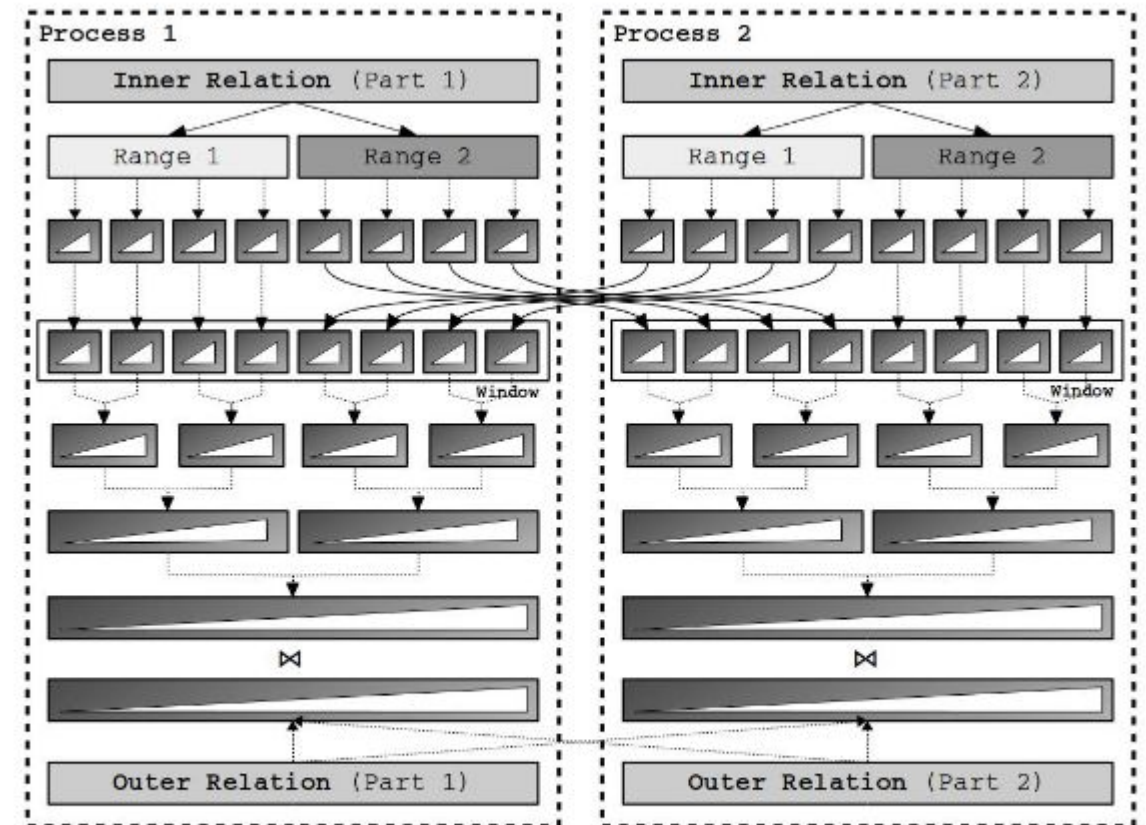


Figure 2: Distributed sort-merge join with two processes.

Goal

Table R			Table S					
Key	Value		Key	Value				
9	A		3	X				
8	B		8	Y				
5	C		4	Z				
1	D	⋈	1	U	=			
10	E		12	V				
12	F		5	W				
6	G		9	S				
11	H		2	T				
						Key	R.Val	S.Val
						1	D	U
						5	C	W
						8	B	Y
						9	F	V

Setup (Initial Input Data)

Suppose the data is initially partitioned on the processes like so.

Process 0

Table R

[(9, A),
(8, B),
(5, C),
(1, D)]

Table S

[(3, X),
(8, Y),
(4, Z),
(1, U)]

Process 1

Table R

[(10, E),
(12, F),
(6, G),
(11, H)]

Table S

[(12, V),
(5, W),
(9, S),
(2, T)]

1. Partition within processes

Partition the data in by range.

- Process 0: keys 1 – 6
- Process 1: keys 7 – 12

Process 0

Table R

[(9, A),
(8, B),
(5, C),
(1, D)]

Table S

[(3, X),
(8, Y),
(4, Z),
(1, U)]

Process 1

Table R

[(10, E),
(12, F),
(6, G),
(11, H)]

Table S

[(12, V),
(5, W),
(9, S),
(2, T)]

1. Partition within processes

Partition the data in by range.

- Process 0: keys 1 – 6
- Process 1: keys 7 – 12

Process 0

Table R	Table S
0: [(5, C), (1, D)]	0: [(3, X), (4, Z), (1, U)]
1: [(9, A), (8, B)]	1: [(8, Y)]

Process 1

Table R	Table S
0: [(6, G)]	0: [(5, W), (2, T)]
1: [(10, E), (12, F), (11, H)]	1: [(12, V), (9, S)]

2. Sort locally

Divide the partitions into “runs” of fixed length (e.g., length 2).

Note: the partitions may not divide evenly into runs.

Process 0

Table R	Table S
0: [(5, C), (1, D)]	0: [(3, X), (4, Z)] [(1, U)]
1: [(9, A), (8, B)]	1: [(8, Y)]

Process 1

Table R	Table S
0: [(6, G)]	0: [(5, W), (2, T)]
1: [(10, E), (12, F)] [(11, H)]	1: [(12, V), (9, S)]

2. Sort locally

Sort each run individually.

Process 0

Table R	Table S
0: [(1, D), (5, C)]	0: [(3, X), (4, Z)]
	[(1, U)]
1: [(8, B), (9, A)]	1: [(8, Y)]

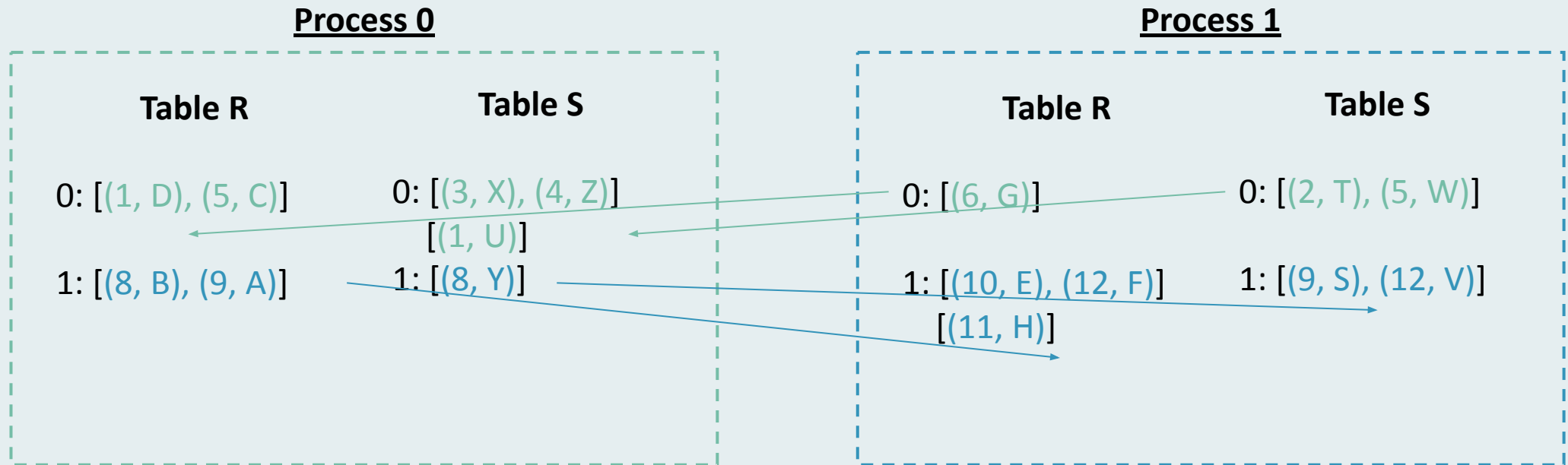
Process 1

Table R	Table S
0: [(6, G)]	0: [(2, T), (5, W)]
1: [(10, E), (12, F)]	1: [(9, S), (12, V)]
[(11, H)]	

3. Exchange across nodes

Send all runs that belong to the other process.

Note: runs are exchanged asynchronously, i.e., the transfer of a run can take place while the process is sorting other runs



3. Exchange across nodes

Send all runs that belong to the other process.

Note: runs are exchanged asynchronously, i.e., the transfer of a run can take place while the process is sorting other runs

Process 0

Table R	Table S
0: [(1, D), (5, C)] [(6, G)]	0: [(3, X), (4, Z)] [(1, U)] [(2, T), (5, W)]
1:	1:

Process 1

Table R	Table S
0:	0:
1: [(10, E), (12, F)] [(11, H)] [(8, B), (9, A)]	1: [(9, S), (12, V)] [(8, Y)]

4. Merge sorted runs

Recursively merge runs.

Process 0

Table R	Table S
0: [(1, D), (5, C), (6, G)]	0: [(1, U), (3, X), (4, Z) [(2, T), (5, W)]
1:	1:

Process 1

Table R	Table S
0:	0:
1: [(10, E), (11, H), (12, F)] [(8, B), (9, A)]	1: [(8, Y), (9, S), (12, V)]

4. Merge sorted runs

Recursively merge runs.

Process 0

Table R	Table S
0: [(1, D), (5, C), (6, G)]	0: [(1, U), (2, T), (3, X), (4, Z), (5, W)]
1:	1:

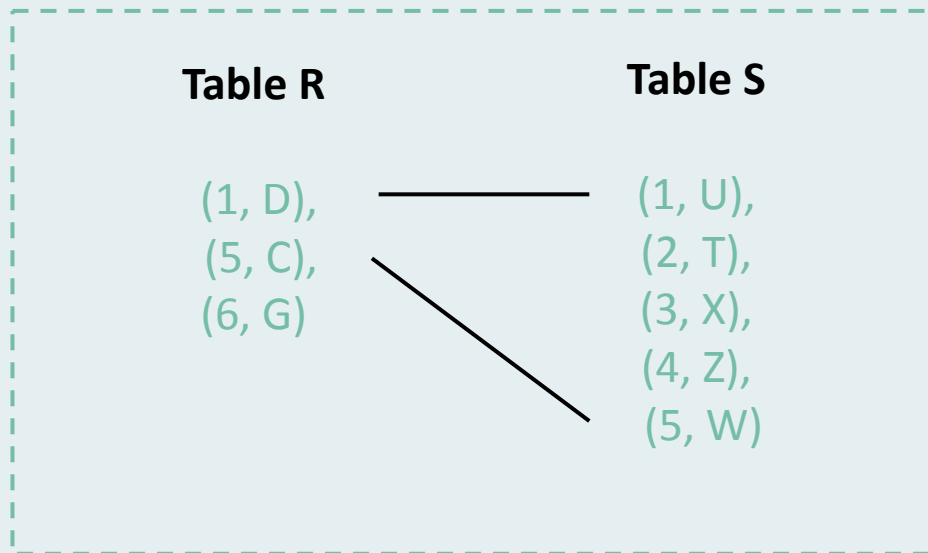
Process 1

Table R	Table S
0:	0:
1: [(8, B), (9, A), (10, E), (11, H), (12, F)]	1: [(8, Y), (9, S), (12, V)]

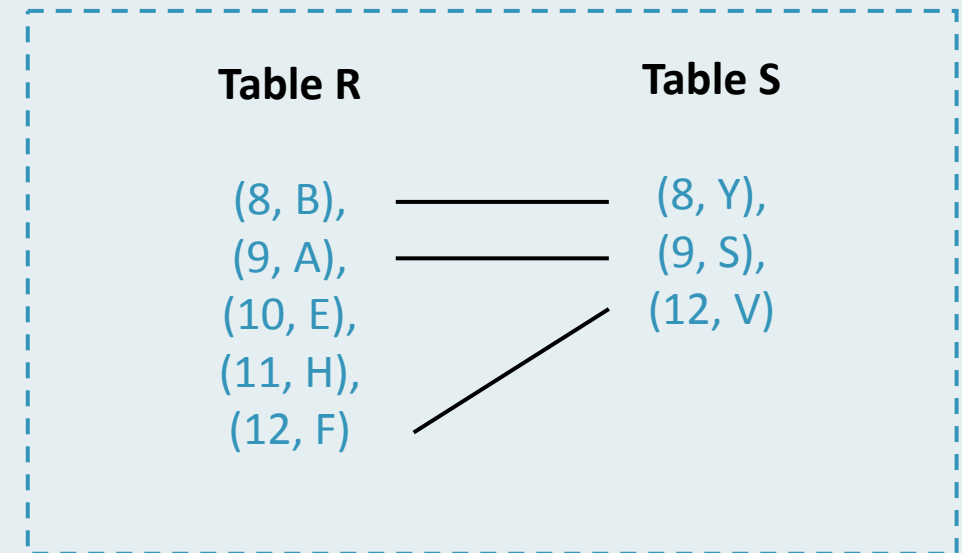
5. Perform local joins

Use the same, non-distributed join method in each process.

Process 0



Process 1



5. Perform local joins

Use the same, non-distributed join method in each process.

Process 0

Key	R.Val	S.Val
1	D	U
5	C	W

Process 1

Key	R.Val	S.Val
8	B	Y
9	F	V

Outline

- Introduction
- Distributed Sort-Merge Algorithm
- **Sort-Merge using MPI**
- Analysis

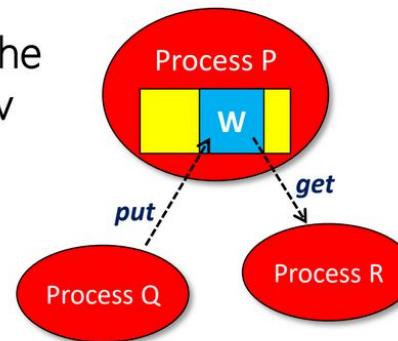
Why use MPI?

- Offers more efficient communication between machines.
- Is hardware-independent but also leverages performance of underlying hardware.
- Provides a good communication infrastructure that selects the most appropriate communication method for each pair of processes.

RMA

Remote memory access

- MPI allows processes to remotely access the memory of another — without send / recv
 - ❖ Process P denotes an area of memory as a "window" W that others can see
 - ❖ Other processes may then "get" data from W or "put" data into W...
 - ❖ See `MPI_Win_create`, `MPI_Put`, and `MPI_Get`



Source: Joe Hummel, Lecture 10

Benefits:

- Reduces CPU load: data doesn't need to be copied into buffers.
- Results in more CPU availability to do processing. Interleave networking and computation components.

Processes write the runs they need to send directly into the target process's window.

Sort-Merge Implementation Steps

1. For each process, create a histogram counting how many tuples belong in each partition.
2. Compute the window size for each process using the histogram.
3. Delegate sections of each window so that writes don't overlap.
4. Sort and transmit runs to corresponding destinations.

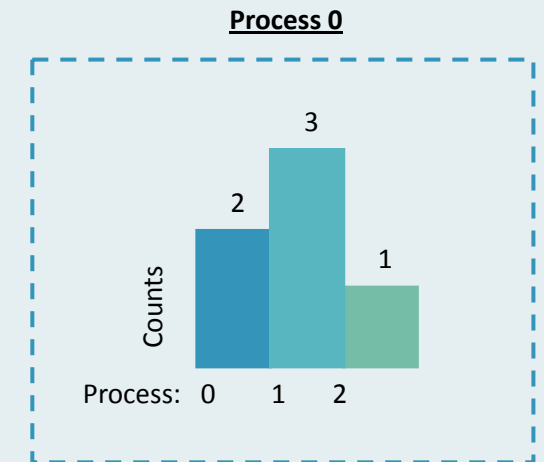
1. Create histograms

For each process, create a histogram counting how many tuples belong in each partition.

This occurs during the partitioning phase of sort-merge (processes keep track of the counts).

If partition is
0: keys 1 – 3
1: keys 4 – 6
2: keys 7 – 9

[(2, A),
(3, B),
(4, C),
(6, D),
(5, E),
(7, F)]



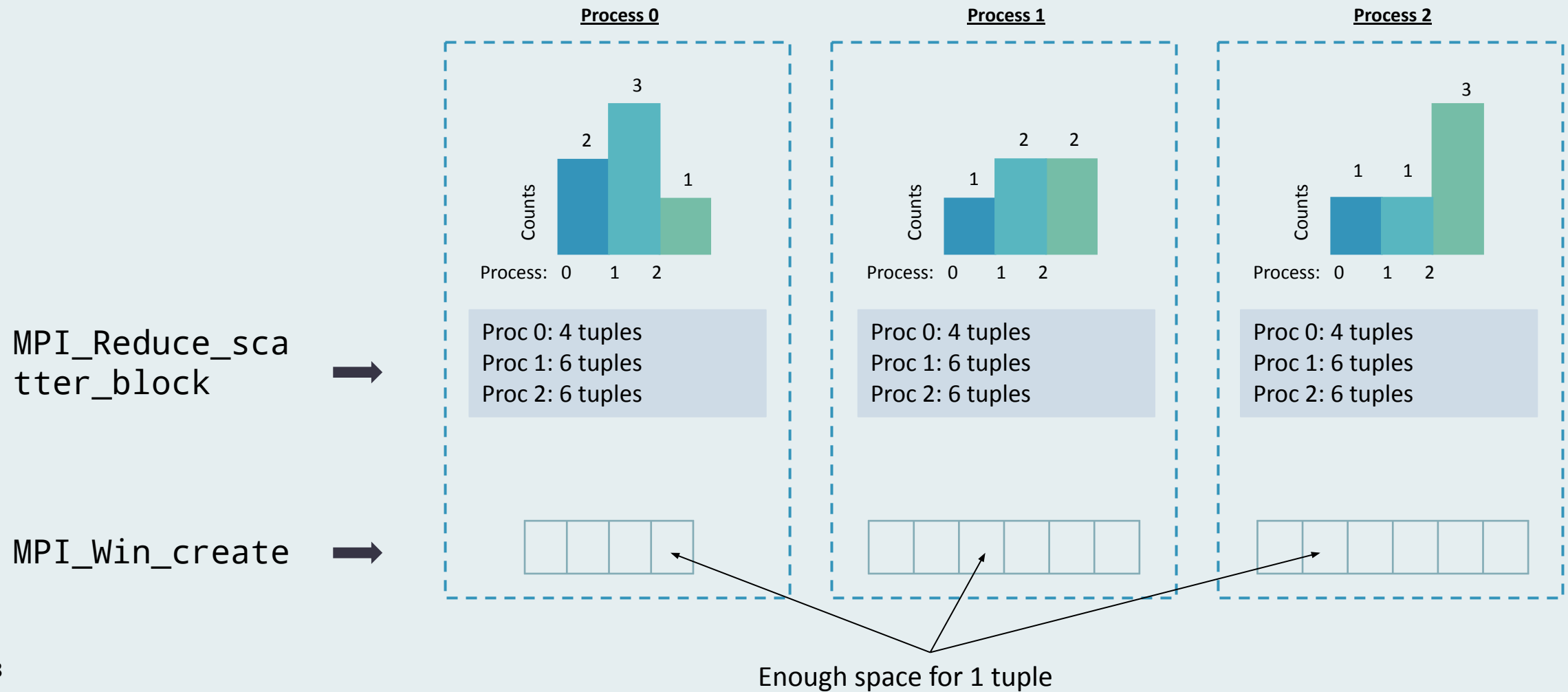
2. Compute the window size

Each process's window must be large enough to store all the rows that fall within the process's assigned partition.

To calculate the window size, use the `MPI_SUM` operation in `MPI_Reduce_scatter_block` and create the windows using `MPI_Win_create`.

- `MPI_Reduce_scatter_block` performs a reduction followed by a scatter. When used with `MPI_SUM`, it performs an element-wise sum across all histograms and scatters the results to the nodes.
- This determines how many tuples each process will receive in its partition.
- For each process, use `MPI_Win_create` to create a window of correct size.

2. Compute the window size

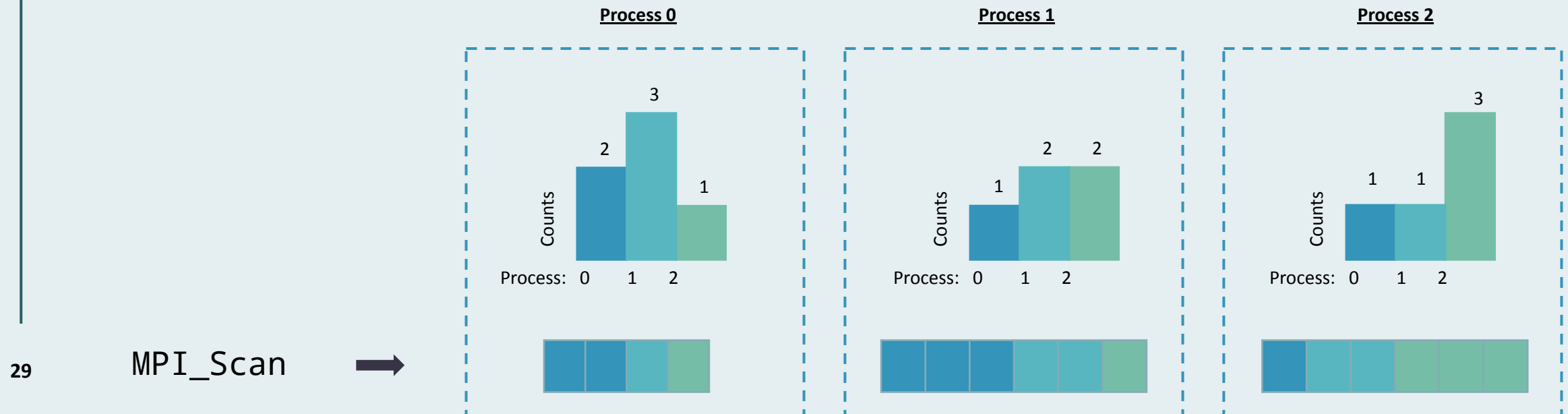


3. Delegate sections of each window

Since we are doing writes to each of the windows, we need to ensure there is no overlap.

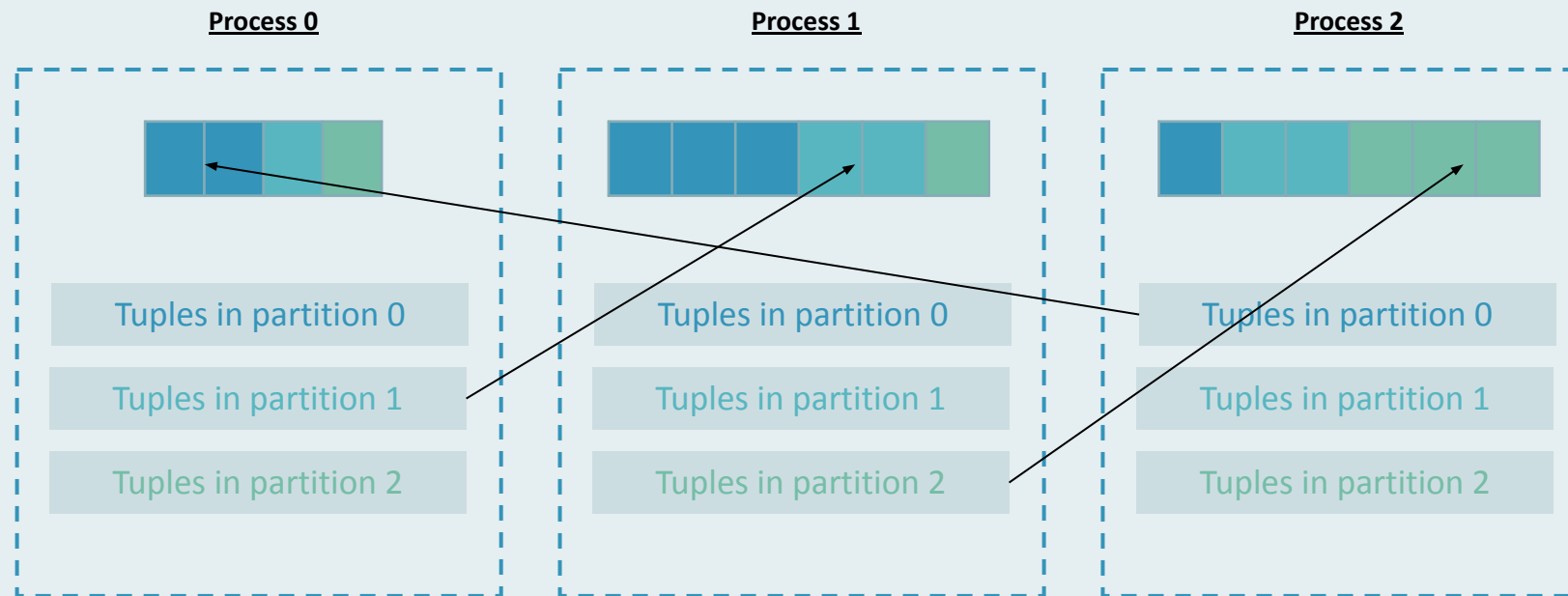
Use MPI_Scan with the MPI_SUM operation to calculate the prefix sums. This gives the total number of tuples contributed by all previous processes.

- MPI_Scan performs a partial reduction.



4. Use MPI_Get and MPI_Put to transmit runs

Since we are doing writes to each of the windows, we need to ensure there is no overlap



Outline

- Introduction
- Distributed Sort-Merge Algorithm
- Sort-Merge using MPI
- **Analysis**

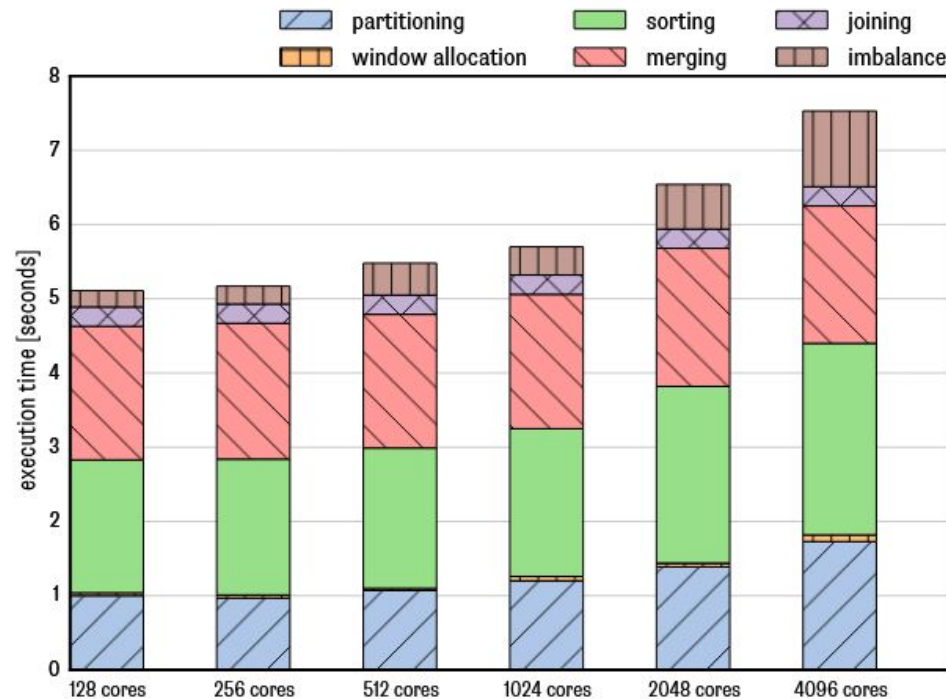
Benefits of Sort-Merge

- Highly parallel
- Computation and communication overlap
- No communication is required during the join itself
- Cache and bandwidth efficient

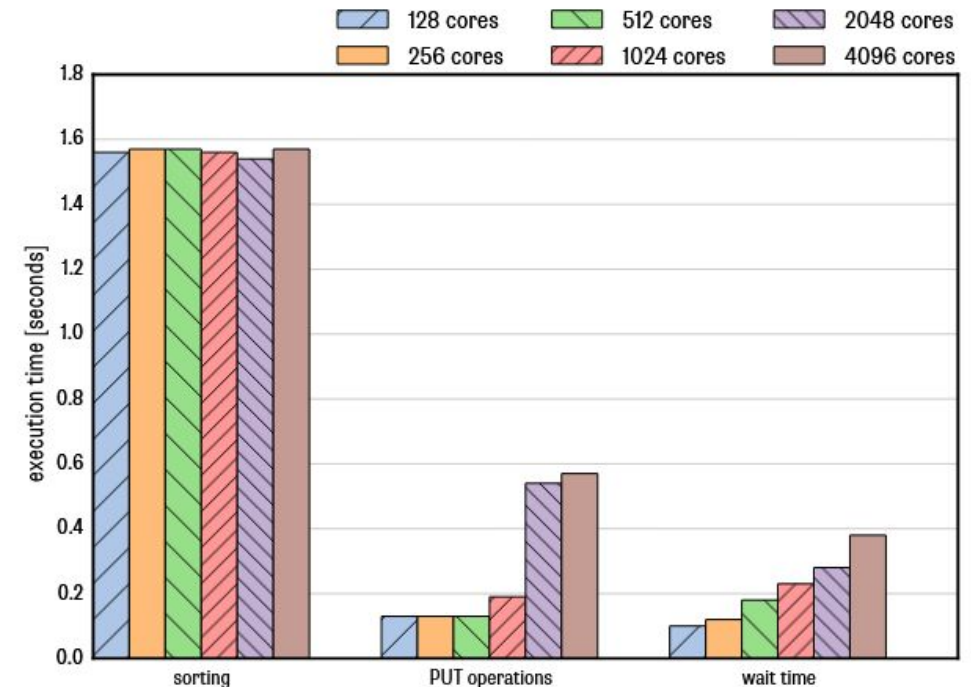
Experiment Setup

- Compare performance on Cray Supercomputers using anywhere from 128 to 4096 cores.
- Ensure input data is evenly and randomly split.

Performance comparison across workloads



(a) Total join execution



(b) Sorting phase

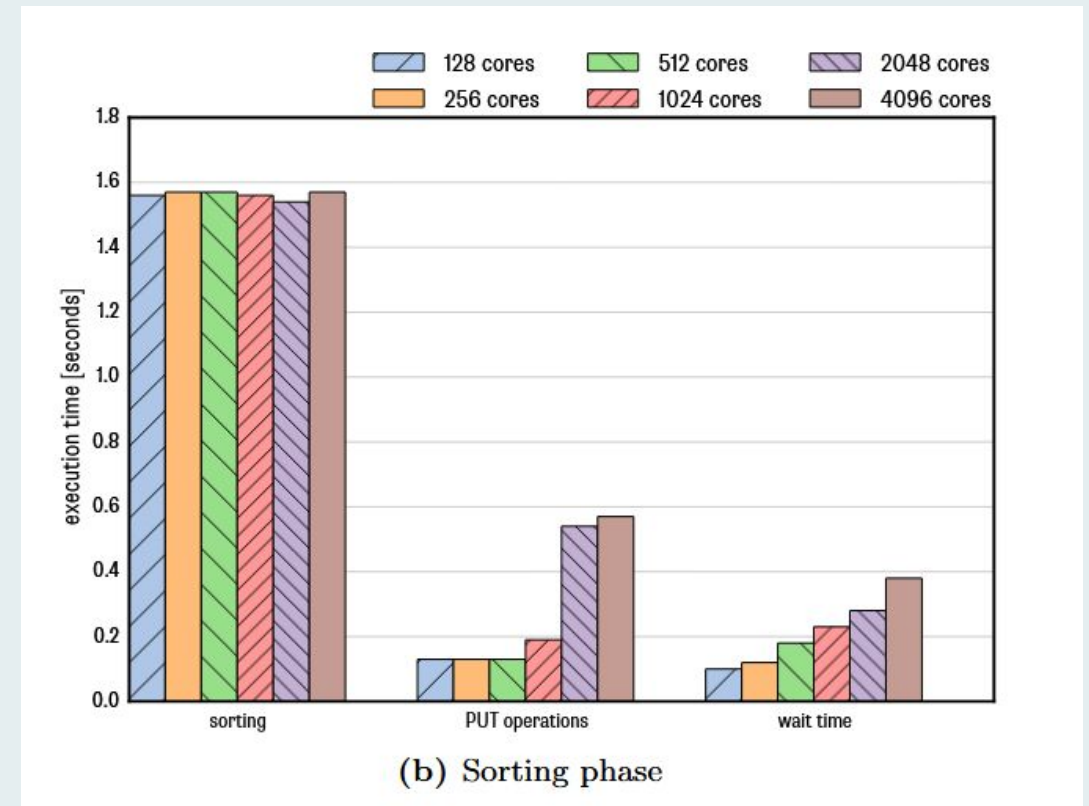
Figure 6: Breakdown of the execution time of the sort-merge join for 40 million tuples/relation/core.

Source: Barthels et al.

Performance comparison across workloads

Note the large jump in PUT operations execution time between 1024 and 2048 cores.

- Sort-merge uses a steady, synchronized communication pattern.
- One run is transferred at a time.
- Processes send data to exactly one peer at a time.
- This only works for small deployments, as larger ones must deal with network latency.



Source: Barthels et al.

Scale-Out vs. Scale-Up Performance

There are two scaling strategies:

- Scale-Out (more nodes) – Cray XC30 (4 cores/node)
- Scale-Up (more cores per node) – Cray XC40 (16 cores/node)

Observations:

- Best performance achieved with fewer cores per node.
- Increasing to 16 caused:
 - More contention on network cards (more processes share a network).
 - Increased time in MPI_Put.
 - Harder to interleave computation and communication.

Overall Takeaways

- Distributed joins are necessary for large-scale data processing.
- The sort-merge join works well because of its parallelism and communication efficiency.
- Network behavior plays a major role in scalability.
- MPI and RMA enable efficient data movement with low CPU overhead.

Thank
you

References

Barthels, Claude, et al. "Distributed join algorithms on thousands of cores." Proceedings of the VLDB Endowment 10.5 (2017): 517-528.